

Robot Agents That Generate Reactive Policies

State Machines, Stateful Programs, and Constraint-Based Execution

Research Proposal II · Zimo Wen

September 2026

Abstract

A robot agent may write an executable plan that becomes obsolete while the robot is acting. Repeated language-model calls can update the plan, but they place inference latency inside the response path. A manually written collection of if–else rules avoids that delay without demonstrating an agent-specific research contribution. This proposal instead asks whether an agent can synthesize a task-appropriate reactive policy whose local execution continuously incorporates new observations. Three representations are compared: a generated state machine, a stateful feedback program, and a generated objective/constraint specification solved locally. All use the same sensors and motion backend. The paper tests whether the generated feedback structure transfers to unseen tasks and changes, reduces unnecessary replanning, and improves actual recovery at a matched computation budget. State machines are one candidate representation within a broader policy-synthesis problem, rather than the full definition of responsiveness.

1. Research Question: Generate a Policy, Not Only an Action List

A motivating difference. An open-loop plan says “move to location A, descend, close the gripper.” A reactive policy says “track the current cup estimate, descend when alignment is sufficient, and close only when the grasp condition is observed.” If the cup moves slightly, local execution updates its target without waiting for another complete agent turn. If the cup disappears or the task changes, the policy identifies an invalid assumption and escalates. The distinction concerns the dependence of actions on observations, not the number of if–else statements.

Agent-specific contribution. A fast controller or a background thread can be useful without being a new Robot Agent method. Here the agent must infer what needs monitoring, which observations change the current behavior, what memory is required, and when its local policy stops being applicable. The generated object is a reusable feedback mapping, not simply a fixed sequence with a more frequent timer. Its value is evaluated on task and disturbance combinations not encoded by a developer in advance.

Problem formulation. A synthesis model maps instruction g , initial permitted observations, and a declared primitive library to a policy P . At local execution step t , P maps observation o_t and memory z_t to an action reference, updated memory, and an explicit status: continue, complete, or escalate. The low-level motion realization is fixed across comparisons. This separates policy representation from the first proposal’s physical action interfaces and from the third proposal’s GPT audit of a VLA candidate.

Existing work. Code as Policies already generates programs that may include feedback. Real-Time Chunking addresses asynchronous execution of learned action chunks; Reactive Diffusion Policy and Force Policy provide slow–fast contact architectures. We cannot claim novelty for feedback loops or asynchronous execution alone. The proposed question is whether *task-conditioned synthesis of feedback structure and applicability conditions* produces better generalization and response than open-loop programs, fixed generic monitors, and developer-written state machines.

Hypotheses. H1: generated task-specific feedback improves disturbed-task completion over fixed sequences at matched sensing and action access. H2: state machines, stateful programs, and constraint policies have different strengths across discrete mode changes, continuous tracking, and geometric replanning. H3: explicit validity and escalation conditions reduce costly full replanning while preserving recovery. If a generic handcrafted monitor performs equally well, the agent-specific claim must be narrowed.

2. Approach A: Agent-Generated State Machines

Policy representation. Represent a policy as states, state-local actions, observable guards, and transitions. For grasping, possible states are observe, approach, align, grasp, lift, and recover. The agent chooses this structure from the task, rather than merely filling in a fixed task-specific graph. A transition might require stable alignment for several observations; recovery might depend on an observed grasp inconsistency. Store timers and short measurement histories where necessary to avoid switching on one noisy frame.

Generation contract. Each guard names its observation source, freshness requirement, threshold units, and action consequence. States specify entry behavior, local update, timeout, and exit. The runtime checks missing transitions, unreachable states, loops without bounded progress, incompatible action ownership, and absent observations. These checks establish structural executability, not task success. Guards cannot call a hidden simulator success predicate or use exact target geometry unavailable through sensors.

Local execution and repair. The state machine consumes a stream of code-backed perception and proprioception at a fixed measured rate. It issues short-horizon references through the common motion backend. An invalid assumption triggers a bounded agent revision of the relevant state or transition; unaffected states are retained. Test-time revision remains within the current episode and is logged separately from development-time skill accumulation. Compare generated graphs with fixed generic graphs and task-specific human graphs.

3. Approach B: General Stateful Feedback Programs

Beyond discrete branches. Some behavior is naturally continuous or depends on history. A program can estimate target velocity, smooth noisy observations, maintain object identity through brief occlusion, and continuously update an approach offset. It need not enumerate every possible target location as a separate state. The agent generates a local function with the common contract

$$(u_t, z_{t+1}, r_t) = P(o_t, z_t), \quad r_t \in \{\text{continue, complete, escalate}\}.$$

Here u_t is a motion reference and z_t stores permitted estimates and timers. The discrete status describes execution management; the action mapping itself can be continuous and stateful.

Example. A tracking policy computes the current grasp approach from the latest observed object pose and a task-specific offset, updates a filtered velocity estimate, and reduces progress when uncertainty rises. It transitions toward grasping only when measured alignment and observation freshness satisfy the policy. Persistent loss of the target produces escalation. The robot does not chase a coordinate frozen at program-generation time.

Implementation boundary. Programs use declared geometry, filtering, perception, and motion functions in a restricted runtime. No arbitrary filesystem access, simulator introspection, or unbounded remote model calls occur inside the local step. Bound memory, step duration, output magnitude, and revision count. If the backend only offers a blocking full-trajectory call, expose a validated short-step interface before claiming continuous response; adding a Python loop around blocking calls does not establish the required behavior.

Comparisons. Remove memory, freeze target updates, and replace task-conditioned logic with a generic tracker while preserving the same data stream. Compare full regeneration with local patching. If performance comes entirely from a stronger perception module or more frequent observations, the program-synthesis contribution has not been isolated.

4. Approach C: Generated Objectives and Constraints

Representation. The agent generates a task-specific cost and observable constraints, then a fixed local solver updates actions as the scene changes. For example, reaching toward a moving target can balance current goal error and motion smoothness while respecting estimated obstacle clearance. A short-horizon controller repeatedly solves

$$\min_{u_{t:t+H}} C_P(\hat{s}_t, u_{t:t+H}) \quad \text{subject to} \quad g_P(\hat{s}_t, u_{t:t+H}) \leq 0,$$

where $\hat{s}_t$ is estimated from permitted observations. The agent determines the task-dependent objectives, constraint sources, and applicability conditions; the solver supplies numerical optimization. No exact hidden obstacle pose enters the constraints.

Why this is a broader policy. New target and obstacle estimates change the optimization directly, without requiring a new if-else branch for every layout. Discrete stage changes can be represented by a small supervisor. Infeasibility, stale observations, or an invalid local dynamics approximation trigger escalation. A complex solver is not assumed superior; compare simple analytic tracking and a fixed generic objective under matched runtime budgets.

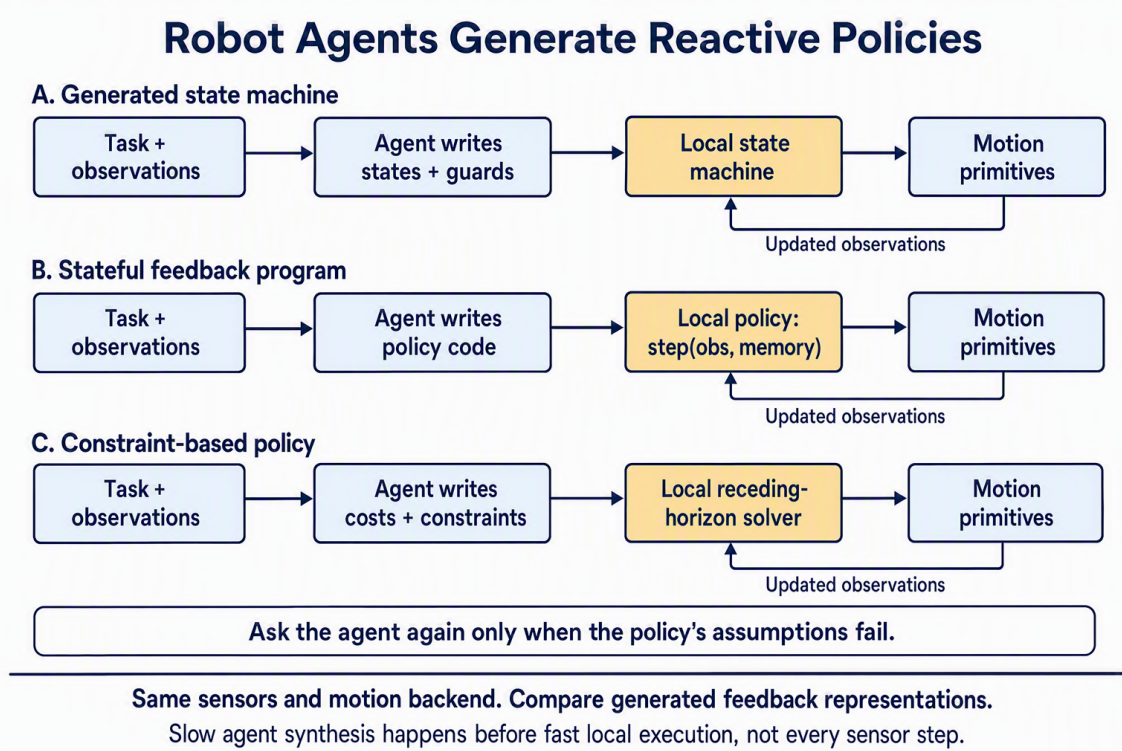


Figure 1: The agent synthesizes one of three feedback representations; updated observations enter its local execution loop. The sensor stream does not require a new language-model call at every step. GPT-image-2 conceptual diagram. All methods share perception and motion primitives.

Common policy lifecycle. Synthesize, validate, initialize memory, execute local updates, check completion or applicability, and request a bounded revision if needed. Record the generated policy version and the observation that justified each revision. Replacing a policy requires a controlled handoff; an obsolete asynchronous result cannot overwrite a newer policy. This lifecycle is shared infrastructure, not three separate higher-performance controllers.

5. Experimental Design: Generalization of Reactive Structure

Task and disturbance families. Use LIBERO short tasks with matched disturbances: target movement during approach, transient occlusion, changed visible path clearance, grasp inconsistency, and instruction or stage changes. Disturbances remain physically meaningful and leave the task solvable. The test set includes unperturbed tasks to measure regressions caused by unnecessary feedback and escalation. Hidden event schedules stay with the evaluator.

Task-level splitting. Group near-duplicate recipes and their language/layout variants before creating development, validation, and test splits. The synthesis model sees the test instruction and initial observations, but does not receive hand-coded rules for its event sequence. Hold out both tasks and task–disturbance combinations. Report whether the primitive library already contains a full task solution; otherwise retrieval of that skill could masquerade as new reactive synthesis.

Proposed panel. Begin with 24 held-out task–disturbance pairs, three strengths, and five seeds: 360 episodes per method. An additional matched subset varies observation or model latency. Determine the actual delay levels from validation relative to the task timescale, rather than copying arbitrary millisecond targets. Task pairs sharing a recipe remain one statistical cluster. The panel size is provisional and must be assessed against development variance.

Comparison	What is generated or fixed	Question isolated
Open-loop program	Fixed action sequence	Value of observation dependence
Repeated agent replanning	New plan at fixed intervals	Response benefit versus inference cost
Generic manual monitor	Fixed event rules and recovery	Value of task-specific synthesis
Task-specific manual FSM	Human-designed states and guards	Strong structural reference
Generated FSM	States, guards, and transitions	Discrete feedback synthesis
Stateful policy program	Continuous actions and memory	Broader feedback representation
Constraint-based policy	Task costs and constraints	Value of local optimization

Fairness. All methods use identical perception outputs, timestamps, short-horizon motion commands, and motion-controller limits. Match local execution rates and separately compare fixed total VLM-call or token budgets. A manual state machine may require more human design effort, which should be documented separately rather than converted into an arbitrary performance penalty. Do not give generated policies additional hidden sensing or stronger task primitives.

Two evaluations. A policy-level diagnostic injects observations into frozen generated policies to inspect guard decisions, memory behavior, and output validity. It does not count as robot task success. Full closed-loop simulation then measures execution under actual changing observations. The standard 120-task LIBERO panel separately reports frozen task-level pass@10 by spatial/object/goal and task/object/swap/lan, without mixing dynamic-extension results into the headline.

6. Planned Ablations and Meaningful Response Metrics

Experiment A: generated feedback versus more rules. Compare generic manual rules, task-specific manual FSMs, and generated policies with a fixed primitive set. Hold code size or synthesis-token budget approximately controlled in a secondary panel, but do not use the number of branches as a capability metric. Test whether generated guards monitor the right task-dependent quantities and whether the resulting correction helps after the event. More conditions can increase false triggers rather than improve response.

Experiment B: discrete, stateful, and optimization-based policies. Evaluate mode changes, continuous target drift, brief occlusion, and changing obstacle geometry separately. Expected strengths are hypotheses: FSMs may suit discrete stages, stateful programs may preserve tracking through ambiguous observations, and constraints may help spatial adjustment. Test these claims rather than selecting only tasks favorable to each representation. Report when a simpler representation achieves equal success at lower synthesis or runtime cost.

Experiment C: memory and applicability. Remove history, smoothing, freshness checks, and escalation conditions separately. Compare local patching with regeneration of the entire policy. Measure whether the agent recognizes an invalid assumption and revises the responsible component, or repeatedly changes unrelated code. Test-time policy versions and their triggers are retained. Failed patches and excessive escalations count against task and resource budgets.

Experiment D: response under delay. Measure the event time, first observation containing it, local policy update, controller acceptance, and actual corrective actuation. Local policies should absorb changes within their declared validity domain without waiting for an agent call. Out-of-domain events require slower reasoning and must be reported separately. Compare with asynchronous action-chunk execution where appropriate, but do not attribute a scheduler’s existing advantage to policy synthesis.

Primary metrics. Report final disturbed-task success, harmful response on unperturbed tasks, false/missed triggers, intervention count, escalation rate, response p50/p95, recovery time, and VLM cost. A program detecting the event but failing to change the robot is not responsive. A program that stops instantly but never resumes has a stop-path benefit, not successful manipulation recovery. Nonresponses remain visible with their timeout rate and censoring, rather than disappearing from latency summaries.

Possible result	Supported conclusion
Generated policies generalize and reduce replanning cost	Supports synthesizing task-dependent feedback structure.
Manual generic rules match performance	The agent-specific contribution is limited in this task set.
Stateful programs help only during occlusion	Evidence for memory in a specific observable regime.
Constraint policies reduce geometric failures but cost more time	A representation–compute tradeoff, not uniform superiority.
More code increases false escalation and regressions	Feedback complexity requires validity and relevance checks.

Use matched seeds, task-clustered confidence intervals, and multiple synthesis samples where generation is stochastic. Freeze prompts and selection rules before testing. Select checkpoints or programs using validation only; success-conditioned cherry-picking of generated code cannot establish deployment reliability.

7. Feasibility, Risks, and Expected Paper Contribution

Robot-RSI integration. The Planner or Engineer generates the policy representation, the runtime validates and executes it over existing perception and motion primitives, and visible traces support bounded revision. The proposal does not require a newly learned force controller. Existing implementations have different cancellation and streaming capabilities, so document the selected backend and hold it fixed across methods. A lock fix or faster transport alone is infrastructure work, not the policy-synthesis contribution.

Limits of verification. A syntactically valid state machine can contain incorrect guards; a terminating program can use an inappropriate recovery; an optimizer can satisfy a badly specified objective. Structural checks must be complemented by actual simulator outcomes. Robot observations are imperfect, so policies need a stated uncertainty/freshness domain. Neither handcrafted guards nor agent-generated constraints are presumed to guarantee safety or task completion.

Preventing hidden-state leakage. The agent, local code, and reviewers receive only declared images, estimated geometry, proprioception, and tool receipts. Hidden task predicates, future events, object ground truth, and private evaluator checkpoints are inaccessible. Learning a guard from a visible signal is allowed; reading the simulator’s answer to decide a transition is not. Post-hoc adjudication remains independent of the generated program’s own completion flag.

Evidence and cost. Preserve generated policies, all revisions, videos, trajectories, failures, and exact manifests. Count success only from final simulator/harness verdicts, with provider, transport, image, resource, and interrupted records reported separately outside valid-verdict denominators. Also show implementation errors, invalid-program rates, and incomplete coverage. Record synthesis and revision tokens, VLM/unmetered calls, wall time, perception/action/recovery time, local-step latency, and solver cost. Efficiency claims require task success to remain acceptable, not merely fewer language-model calls.

Expected contribution. The paper should establish whether a robot agent can turn task understanding into a feedback policy that continues to work as observations change. Its methods compare a finite-state representation with more general stateful and constraint-based policies; its experiments isolate generation from stronger sensing and control. The central result is generalization of the reactive structure under a fixed execution backend. This is different from the first paper’s action/force representation and the third paper’s retained GPT audit of VLA proposals.

Scope of the claims. Feedback programs, hybrid systems, local optimization, and asynchronous policies already exist. A new claim must concern how the agent synthesizes, grounds, and revises them, and what measurable benefit this yields. If fixed manual policies suffice, report that outcome. The proposal contains experimental designs, not newly measured success rates or promises of hard real-time execution.

References

- [1] Code as Policies: Language Model Programs for Embodied Control. 2022. <https://arxiv.org/abs/2209.07753>
- [2] Real-Time Execution of Action Chunking Flow Policies. 2025. <https://arxiv.org/abs/2506.07339>
- [3] Reactive Diffusion Policy: Slow-Fast Visual-Tactile Policy Learning for Contact-Rich Manipulation. 2025. <https://arxiv.org/abs/2503.02881>
- [4] Force Policy: Learning Hybrid Force-Position Control Policy under Interaction Frame for Contact-Rich Manipulation. 2026. <https://arxiv.org/abs/2602.22088>
- [5] Noematrix Team. RoboRSI research preview. 2026. <https://lab.noematrix.ai/blog/2-roborsi-research-preview/>